

Programmatoid and neuronoid computations

Nicolas Rougier et al

June 26, 2026

What about ?

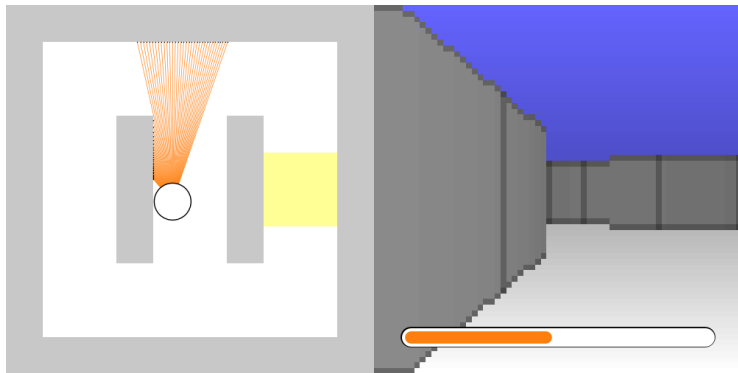
Programming (not training) neural networks

- ▶ With a benchmark and ...
- ▶ Using computer algebra.

But ... why ?

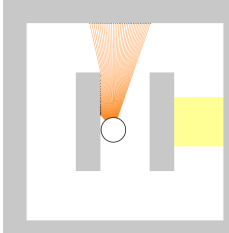
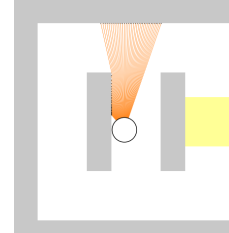
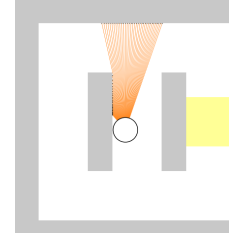
- ▶ For the fun :)
- ▶ To illustrate a feed-forward neural network computational property.
- ▶ To provide an approximate neural network complexity lower bound, given a task.
- ▶ To obtain a parsimonious computation tool.

The braincraft benchmark

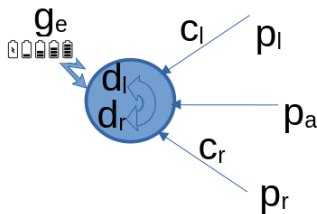


Documented in the README ☒ (please).

Three tasks:

 The diagram shows a robot (a small white circle) in the center of a corridor. The corridor has a gray floor and white walls. To the left and right of the robot are two gray vertical bars. To the right of the robot, there is a yellow rectangular area representing an energy source. Orange lines radiate from the robot, indicating its field of view or sensor range.	 The diagram is identical to the first one, showing a robot in a corridor with a yellow energy source to its right.	 The diagram is identical to the first one, showing a robot in a corridor with a yellow energy source to its right.
Find the energy source leftward or rightward ✓.	Find the correct direction, given the blue cue ✓.	Choose the best energy source ✓.

A (simplified) bot, with:

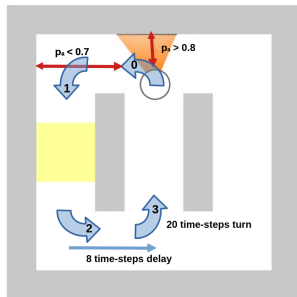
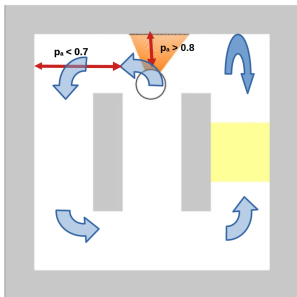


- left, ahead and right proximity sensors

$$p_{\bullet} \in]0_{\text{no-wall}}, 1_{\text{wall-hit}}], \bullet \in \{l, a, r\},$$

- color sensors $c_{\bullet\uparrow}$,
- energy gauge $g_e \in [0_{\text{death}}, 1_{\text{full}}]$,
- running at constant velocity, and
- turning leftward or rightward $d_l \in [0, 1_{+5^\circ}]$.

Heuristics:



- ▶ Stabilizes the forward motion using linear feedback correction.
- ▶ Performs quarter-turn using ahead proximity thresholds.
- ▶ Performs about-turn or half-corridor turn using delays.
- ▶ Stores the preferred quarter-turn direction, e.g. from cue.
- ▶ Stores previous energy feeds for comparison.

Programmatic implementation:

```
class State:
    """ Implements a programmatoid state.
    """

    data = dict()
    """ The state data: dictionary.
        - Stores all input, output, and internal variables, including parameters.
    """

    def init(self):
        """ Inits some data, if needed.
        """

    def update(self):
        """ Updates, at each step, the data from input and sets the output value.
        """
```

- ▶ About 8 internal variables
(forward/turns states, turn direction, delays, gauge values)
- ▶ About 40 lines of straight-line program code
(beyond initialization and declarations)
 - when using about-turn ☑ and
 - about 50 when using quarter-turns only ☑.

So what ?

From programmatic
to programmatoid and neuronoid
implementation.

The notion of “programmatoid computation”:

- It is an input-output straight-line program $\checkmark$ implementing an operator on either
 - binary values $\in \{0, 1\}$
 - or continuous normalized values $\in [0, 1]$,

which evolution is defined using an LN-system with linear units or the step-function $H(x) \in \{0_{x<0}, 1/2_{x=0}, 1_{x>0}\}$ as non-linearity:

$$\begin{aligned} o_n[t] &= \begin{cases} H(z_n[t]) \\ z_n[t] \end{cases}, \\ z_n[t] &\stackrel{\text{def}}{=} \sum_{m \in \{1, M\}} w_{nm} i_m[t-1] + w_{n0}, n \in \{1, N\} \end{aligned}$$

where $i_m[t]$ is the m -th input at a discrete t , and $o_n[t]$ an output.

- The key idea:

programmatoid implements conditional expressions
on binary or numerical values:

$$\begin{aligned} \text{value} &= \text{if condition then true_value else false_value} \\ &\quad \longrightarrow \\ \text{value} &= H(\text{condition}) \text{ true_value} + H(\text{not condition}) \text{ false_value} \end{aligned}$$

For a property $\mathcal{P}$, $H(\mathcal{P}) \in \{0_{\mathcal{P} \text{ is false}}, 1_{\mathcal{P} \text{ is true}}\}$.

The notion of “sugared straight-line program”:

- It is an input-output piece of code with
 - instruction sequences, assignments, conditional instructions, and iterations (no general loops);
 - using [generalized] programmatoid computation in assignment.
- ▶ This implements the set of primitive recursive functions $\checkmark$.
- ▶ Any primitive recursive functions rewrites as a generalized programmatoid:
 - ▶ A generalized programmatoid computation includes:
 - exponential and logarithm as non-linearity, when not
 - other p $\checkmark$ ure function.

The reasons why (not a formal proof):

- ▶ Iterations unfolding

```
for n from 0 to n.1 ≤ n.max do
  instructions.1 endfor
```

→

```
if n ≥ 0 then
  substitute(n = 0, instructions.1)
  ...
if n = n.1 then
  substitute(n = n.1, instructions.1)
```

```
if condition.1 then instruction.1 instructions.2
elif condition.2 then instructions.3
...
else instructions.4 endif
```

→

```
if condition.1 then instruction.1 endif
if condition.1 then instructions.2 endif
if not condition.1 and condition.2 then instructions.3 endif
...
if not condition.1 and not condition.2 then instructions.4 endif
```

```
if condition then name = value endif
```

→

```
name = if condition then value else name
```

- ▶ Conditional expansion

- ▶ Conditional reduction

- ▶ Introducing $\log(\cdot)$ and $\exp(\cdot)$ makes products, thus polynomials, implementable.

- ▶ Introducing polynomial, makes regular functions approximatable.

The notion of “neuronoid”:

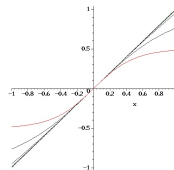
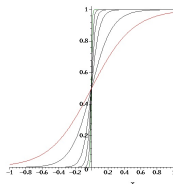
This names the not very biologically plausible simplest biological neuron inspired by mean-field modeling of the Hodgkin–Huxley neuronal axon model:

$$\tau \frac{\partial v_i}{\partial t}(t) + v_i(t) = z_i(t), z_i(t) \stackrel{\text{def}}{=} h\left(\sum_j w_{ij} v_j(t) + w_{i0}\right),$$

using the normalized sigmoid with:

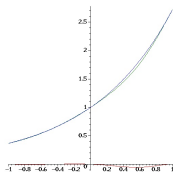
$$h(-\infty) = 0, h(0) = 1/2, h'(0) = 1, h(+\infty) = 1, h(x) = 1 - h(-x).$$

- ▶ A neuronoid $z \rightarrow h_{\tau=0}(\omega z)|_{\omega > 10}$ approximates $H(\cdot)$ below 10^{-10} .
- ▶ A neuronoid $z \rightarrow \omega h_{\tau=0}(\frac{z}{\omega})|_{\omega > 100}$ approximates $z \rightarrow z$ below 10^{-3} .

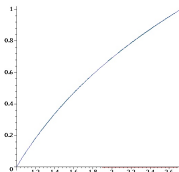


Further use of “programmatoid/neuronoid”:

Combinations of neuronoids $\tau = 0$ approximate regular functions:



The $\exp(\cdot)$ function below 10^{-3}



The $\log(\cdot)$ function below 10^{-2}

$$o \stackrel{\text{def}}{=} g \left(\underbrace{\frac{1}{K} \sum_k i_k}_{\text{mean}} \right) + (1 - g) \left(\underbrace{\sum_k b_k i_k}_{\text{max}} \right)$$

for b_k based on $H(i_h > i_l)$ combinations.

The softmax operator

Neuronoids with $\tau > 0$ implements temporal operators:



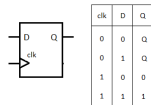
Delay operator



Bistable operator



Astable operator



D latch

Implemented functions 1/3

For binary variables $b_{\bullet} \in \{0, 1\}$, continuous variables $v_{\bullet} \in [-1, 1]$, and the sigmoid $h(\cdot)$ the following functions, defined previously, are implemented:

$v_o = h(v)$	The normalized sigmoid.
$v_o = H(v)$	Generalized step-function. $v_o \in \{0_{v<0}, 1/2_{v=0}, 1_{v>0}\}$ Converted to $h(\cdot)$ when the option <code>prgm["neuronoid"] = true</code> .
$v_o = Id(v)$	Identity function Converted to a neuronoid when the option <code>prgm["neuronoid"] = true</code> .
$b_o = And(b_1, \dots)$	Conjunction with a variable number of binary arguments. Implements b_1 and $b_2 \dots$
$b_o = Or(b_1, \dots)$	Disjunction with a variable number of binary arguments. Implements b_1 or $b_2 \dots$
$b_o = Not(b)$	Negation of a binary value. Implements <code>not b</code>
$b_o = If_b(b_1, b'_1, \dots, b'_0)$	Binary conditional expression, Implements <code>if b_1 then b'_1</code>
$v_o = If_v(b_1, v_1, \dots, v_0)$	Valued conditional expression, Implements <code>if b_1 then v_1</code>
$v_o = Exp_v(v_1)$	Exponential function approximation Implements $v_1 \in [-1, 1], e^{v_1} \in [1/e, e] \simeq [0.368, 2.718]$
$v_o = Log_v(v_1)$	Natural logarithm approximation Implements $v_1 \in [1, e], \log(v_1) \in [0, 1]$
$v_o = Prod_v(v_1, v_2)$	Multiplication approximation Implements $v_i \in [-1, 1], i \in \{1, 2\}, v_1 v_2 \in [-1, 1]$
$v_o = Prod_b(b_1, v_1, \dots)$	Sum of binary weights values Implements $b_1 \quad v_1 + \dots$ Default missing value is 0.

Implemented functions 2/3

<code>v_o = Softmax(v_1, ..., G)</code>	Mean-max operator. $G \in [0_{\text{maximum}}, 1_{\text{average}}]$ controls the mean-max balance.
<code>Latch_b(b_o, b_i, b_c)</code>	Binary memory latch. b_o is the output, b_i is the input, $b_c \in \{0_{\text{open}}, 1_{\text{latched}}\}$ is the control.
<code>Latch_v(b_o, v_i, b_c)</code>	Valued memory latch. b_o is the output, v_i is the input, $b_c \in \{0_{\text{open}}, 1_{\text{latched}}\}$ is the control.
<code>Bistable(b_o, b_1, b_0)</code>	Two inputs bi-stable latch. b_o is the output, b_0 resets to 0, b_1 sets to 1.
<code>Bistable(b_o, b_i)</code>	One input bi-stable latch. b_o is the output, b_i is the two-way input.
<code>Spikeup(b_o, b_i)</code>	Spike generation on input rising front. b_i is the input, which rising front is detected.

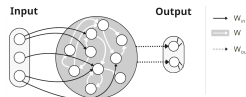
Implemented functions 3/3

<code>Delay(b_o, b_i, T)</code>	Basic delayed output. $T > 0$ is the delay, in number of global clock event, <code>b_o</code> is the output, <code>b_i</code> is the input, that must remains to 1.
<code>Delay_0(b_o, b_1, b_0, T)</code>	Delayed output. $T > 0$ is the delay, in number of global clock event, <code>b_o</code> is the output, <code>b_1</code> is the input, <code>b_0</code> is the optional reset.
<code>Delay_1(b_o, b_1, b_0, T)</code>	Time bounded output. $T > 0$ is the delay, in number of global clock event, <code>b_o</code> is the output, <code>b_1</code> is the input, <code>b_0</code> is the optional reset.
<code>Oscillator(b_o, b_c, T)</code>	Oscillatory output. $T > 0$ is the period, in number of global clock event. <code>b_o</code> is the output, <code>b_c</code> $\in \{0_{stop}, 1_{run}\}$ is the control.

The programmatoid to neuronoid translator:

```
# Task1,2,3 Implementation as a programmatoid
subs()
## Constants values
gamma = 0.025,    ## Linear feedback gain in forward node
alpha = 5,        ## Maximal turn angle
beta_1 = 0.5,     ## Proximity turn trigger threshold
beta_0 = 0.7,     ## Proximity turn stop threshold
delta = 40,       ## About-turn number of steps
[
  prog_options = { challenge = 0, networking = false },
  # Bot state control
  b_d = If(b.And(challenge = 2, Not(b.k), <_l), 1, b.di),
  b_h = If(b.Or(challenge != 2 or c_lb or c_rb), 1, b.k),
  b_l = And(Not(b_l), p_a > 0.8),
  b_r = If(b.If(b.challenge = 3, And(q_d1 > 0, q_e - q_w1) > q_d1),
    And(q_w1 > 0, q_e - q_w1), 1, b_w),
  b_d = If(b.And(b_l, b_w), Not(b_d), b_d),
  Delay_0(b_w, And(b_l, b_w), delta),
  b_0 = And(b_l, If(b_w, b_w, p_a < 0.7)),
  b_t = If(b_0, 1, b_0, 0, b_t),
  b_w = And(Not(b_l), b_0),
  ## Energy variables update
  q_d1 = If_v(And(b_w, q_w1 > 0, q_e > q_w1), q_e - q_w1, q_d1),
  q_w1 = If_v(b_w, q_e, q_w1),
  ## Navigation equations
  d_l = If_v(b_t, If_v(b_d, 5, 8), 0.035 * p_r),
  d_r = If_v(b_t, If_v(b_d, 0, 4), 0.035 * p_l)
];
```

```
class State:
    """ Implements a programmatoid state.
    """
    data = dict()
    """ The state data: dictionary.
        - Stores all input, output, and internal variables, including parameters.
    """
    def init(self):
        """ Sets some data, if needed.
        """
    def update(self):
        """ Updates, at each step, the data from input and sets the output value.
        """
```



► From programmatoid equations ...

► to Python code ...

a straight-line program with $h(\cdot)$ or $H(\cdot)$ non-linearities, if any.

► to neural-network weights values ...

using $h(\cdot)$ non-linearities for all units.

The input syntax uses Maple $\surd$ syntax and the translator $\surd$ is implemented in this language, since based on computer algebra symbolic computing.

Open science approach and experimental reproducibility:

- ▶ Everything is available online
 - under the CeCILL-C  (for the code) and
 - under CC-BY  (for other resources) licenses.
- ▶ All sources are available in a git  repository.
- ▶ All results are available via a single web-page .
- ▶ All commands are gathered in fully documented makefile 
 - ▶ The reproducibility  test, simply writes:
`make rebuild.`
 - ▶ The test's screen casts are recorded.
 - ▶ The complete process trace is made available,
in wJSON , easily readable.

